

Vision par ordinateur

TP 2 – Réseaux de neurones convolutifs (Partie2)

Exercice 1 : Implémentation d'une Variante de VGG pour la Classification d'Images

Dans cet exercice, vous allez implémenter une variante simplifiée du modèle VGG (Visual Geometry Group) pour la classification d'images à l'aide du jeu de données CIFAR-10.



Contenu du dataset

- Nombre total d'images : 60 000
 - 50 000 pour l'entraînement
 - 10 000 pour le test
- Taille des images : 32×32 pixels
- Canaux : 3 (RVB / couleurs)
- Format : numpy arrays de type uint8, valeurs entre 0 et 255

Objectifs de l'exercice

1. Charger et prétraiter les données CIFAR-10.
2. Construire une variante simplifiée du modèle VGG.
3. Compiler et entraîner le modèle.
4. Évaluer les performances du modèle sur les données de test.
5. Visualiser les courbes d'apprentissage pour analyser l'entraînement.

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

# -----
# 1. Chargement et prétraitement des données CIFAR-10
# -----
(train_images, train_labels), (test_images, test_labels) = datasets.cifar10.load_data()
```

```

# Normalisation des pixels
train_images, test_images = train_images / 255.0, test_images / 255.0

# Séparation d'un jeu de validation à partir du jeu d'entraînement (20%)
train_images, val_images, train_labels, val_labels = train_test_split(
    train_images, train_labels, test_size=0.2, random_state=42
)
# -----
# 2. Définition du modèle VGG-like
# -----
model = models.Sequential([
    # Bloc 1
    layers.Conv2D(32, (3,3), activation='relu', padding='same', input_shape=(32,32,3)),
    layers.Conv2D(32, (3,3), activation='relu', padding='same'),
    layers.MaxPooling2D((2,2)),
    layers.Dropout(0.25),

    # Bloc 2
    layers.Conv2D(64, (3,3), activation='relu', padding='same'),
    layers.Conv2D(64, (3,3), activation='relu', padding='same'),
    layers.MaxPooling2D((2,2)),
    layers.Dropout(0.25),

    # Bloc 3
    layers.Conv2D(128, (3,3), activation='relu', padding='same'),
    layers.Conv2D(128, (3,3), activation='relu', padding='same'),
    layers.MaxPooling2D((2,2)),
    layers.Dropout(0.25),

    # Classification
    layers.Flatten(),
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax')
])
# -----
# 3. Compilation du modèle
# -----
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
# -----
# 4. Entraînement du modèle
# -----
history = model.fit(
    train_images, train_labels,
    epochs=20,
    batch_size=64,
    validation_data=(val_images, val_labels)
)

```

```

# -----
# 5. Évaluation finale sur le jeu de test
# -----
test_loss, test_acc = model.evaluate(test_images, test_labels)
print(f'Test accuracy: {test_acc:.4f}')

# -----

# 6. Visualisation des courbes d'apprentissage
# -----
# Accuracy
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.ylim([0,1])
plt.legend(loc='lower right')
plt.title('Training and Validation Accuracy')
plt.show()

# Loss
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend(loc='upper right')
plt.title('Training and Validation Loss')
plt.show()

```

Exercice 2 : Classification binaire d'images *chat* vs *chien* par transfert learning



Dataset Cats vs Dogs (TensorFlow Datasets)

- Nom dans TFDS : "cats_vs_dogs"
- Classes :
 - 0 = Cat
 - 1 = Dog
- Nombre total d'images : 23 262 images couleur ($\approx 11\,500$ chats, $\approx 11\,700$ chiens).
- Dimensions : les images sont de tailles variées (il faut les redimensionner, ex : 224x224).
- Taille du dataset : ≈ 800 Mo.
- Pas encore séparé en train/test $\rightarrow$ il faut le découper manuellement (ex : 80% train, 10% validation, 10% test).

Objectif :

Développer un modèle de classification binaire capable de distinguer les images de **chats et chiens** en utilisant le **transfert learning** avec plusieurs architectures pré-entraînées pour comparer leurs performances.

Étapes du l'exercice :**1. Préparation des données**

- Utiliser un dataset standard comme **Cats vs Dogs**.
- Redimensionner les images et les normaliser selon les exigences des modèles pré-entraînés.
- Diviser le dataset en **train, validation et test**.

2. Choix des architectures pour comparaison

- Importer plusieurs modèles pré-entraînés sur **ImageNet**, par exemple :
 - **VGG16**
 - **MobileNetV2**
 - **ResNet50**
 - **EfficientNetB0**
- Geler les couches convolutionnelles pour utiliser les features déjà apprises.
- Ajouter des couches fully connected pour la classification binaire.

3. Entraînement du modèle

- Définir la fonction de perte : **binary crossentropy**.
- Choisir un optimiseur adapté (Adam, RMSprop) et un learning rate approprié.
- Appliquer **data augmentation** pour augmenter la diversité des images et éviter le surapprentissage.
- Utiliser **early stopping** et **checkpointing** pour optimiser l'entraînement.

4. Évaluation

- Évaluer les modèles sur le jeu de test avec les métriques suivantes :
 - **Accuracy**
 - **Précision, Rappel, F1-score**
 - **ROC Curve et AUC** pour la classification binaire
- Comparer les performances entre **VGG16, MobileNetV2, ResNet50 et EfficientNetB0**.

5. Analyse

- Visualiser les **feature maps** ou les filtres appris pour comprendre ce que chaque modèle détecte.
- Identifier les erreurs de classification et proposer des améliorations.
- Comparer les modèles en termes de **précision, vitesse et taille**.